

Funções e Bibliotecas em Python



Preparação de alunos do 3.º ano (M/FZ)



Apoio ao novo ciclo de estudos —
Mestrado em Ciências Militares Navais





Aula Teórico-Prática

Funções, Lambda e Bibliotecas em Python



Ideia principal

Apresentar os conceitos fundamentais para criar e utilizar funções em Python, incluindo lambda, validação, tratamento de erros e bibliotecas simples.

Área		Conteúdos
	Funções	def, chamada, parâmetros e argumentos
	Procedimentos e funções	print() vs return
	Validação e erros	if, raise, try/except
	Mutabilidade	objetos mutáveis e imutáveis
	Lambda	map(), filter() e programação funcional
	Bibliotecas	math, random e datetime



Organizar

Código mais estruturado e legível



Validar e tratar

Prevenir erros e aumentar a robustez



Reutilizar

Evitar repetição e poupar tempo



Analisar

Transformar dados em informação útil



Explorar

Usar bibliotecas para ir mais longe



Objetivo da aula: dominar conceitos essenciais para escrever código eficiente, seguro e reutilizável em Python.



Aula Teórico-Prática

Funções, Lambda e Bibliotecas em Python



Ideia principal

Apresentar os conceitos fundamentais para criar e utilizar funções em Python, incluindo lambda, validação, tratamento de erros e bibliotecas simples.

Área		Conteúdos
	Funções	def, chamada, parâmetros e argumentos
	Procedimentos e funções	print() vs return
	Validação e erros	if, raise, try/except
	Mutabilidade	objetos mutáveis e imutáveis
	Lambda	map(), filter() e programação funcional
	Bibliotecas	math, random e datetime



Organizar

Código mais estruturado e legível



Validar e tratar

Prevenir erros e aumentar a robustez



Reutilizar

Evitar repetição e poupar tempo



Analisar

Transformar dados em informação útil



Explorar

Usar bibliotecas para ir mais longe



Objetivo da aula: dominar conceitos essenciais para escrever código eficiente, seguro e reutilizável em Python.



1. Introdução às funções



Ideia principal

Uma função é um bloco de código reutilizável que ajuda a organizar o programa, evitar repetição e executar uma tarefa sempre que necessário.



Para que servem?

Utilidade	Explicação
 Evitar repetição	Reutilizar o mesmo código várias vezes
 Organizar o programa	Dividir o código em partes mais simples
 Melhorar a leitura	Tornar o código mais claro
 Facilitar alterações	Alterar uma tarefa num único local



Exemplo em Python

```
def mostrar_mensagem():  
    print("Bem-vindo à aula de Python!")  
  
mostrar_mensagem()
```



Definição da função

A função é criada com a palavra-chave `def`.



Chamada da função

A função é executada pelo seu nome.



Mensagem-chave

Uma função permite reutilizar código, organizar o programa e evitar repetição.





2. Objetos mutáveis e imutáveis



Ideia principal

Em Python, os valores são objetos. O comportamento depende de o objeto ser mutável ou imutável.

 Tipo	 Explicação	 Exemplos
 Imutável	Não pode ser alterado diretamente depois de criado.	<code>int</code> <code>float</code> <code>str</code> <code>bool</code> <code>tuple</code>
 Mutável	Pode ser alterado depois de criado.	<code>list</code> <code>dict</code> <code>set</code>

Comportamento		 Ideia essencial
 Imutável		A alteração faz a variável referenciar outro objeto. 
 Mutável		O conteúdo do próprio objeto pode ser alterado. 



Porque é importante?

Esta distinção é relevante quando os objetos são atribuídos a variáveis ou passados para funções.



Imutáveis

Cada alteração cria/obtem outro objeto.



Mutáveis

Todas as variáveis que referenciam o mesmo objeto partilham as alterações.



Resultado

Comportamentos diferentes no programa.



Mensagem-chave: Entender a mutabilidade ajuda a escrever código correto, eficiente e previsível.





Exemplo: imutável vs mutável



Ideia principal

Em Python, os valores são objetos.
O comportamento depende de o objeto ser mutável ou imutável.



Objeto imutável

```
numero = 10  
mesmo_numero = numero
```

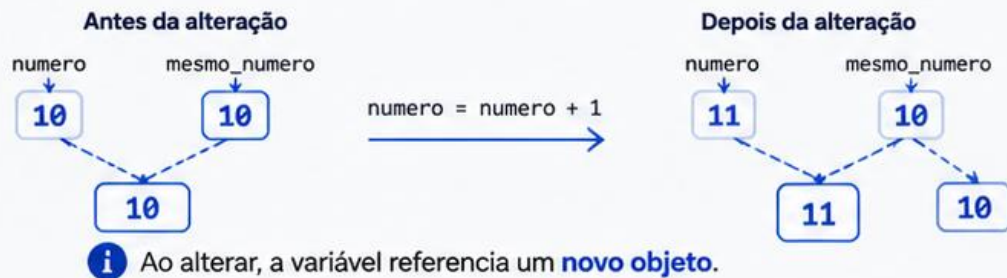
```
numero = numero + 1
```

```
print(numero)      # 11  
print(mesmo_numero) # 10
```

Saída

11
10

Como funciona (imutável)



Objeto mutável

```
nomes = ["Ana", "João"]  
mesma_lista = nomes
```

```
mesma_lista.append("Maria")  
nomes.append("Pedro")
```

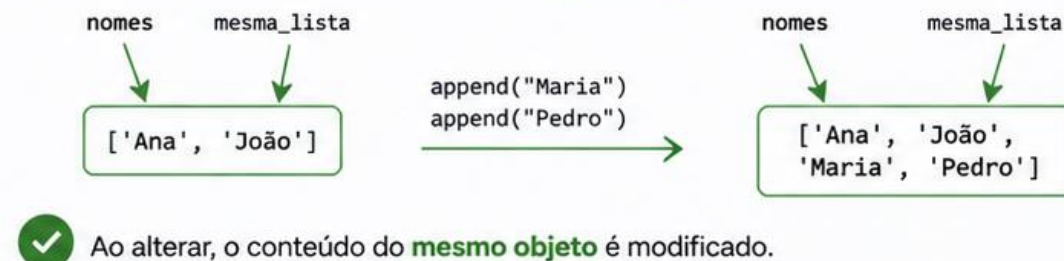
```
print(nomes)  
print(mesma_lista)
```

Saída

['Ana', 'João',
'Maria', 'Pedro']

['Ana', 'João',
'Maria', 'Pedro']

Como funciona (mutável)



Resumo rápido

Diferença essencial
entre os dois comportamentos.



Imutável

O valor não muda,
muda a referência.



Mutável

O valor pode mudar,
a referência permanece.



Referência

Variáveis armazenam referências
para objetos, não os valores.



Por que importa?

Evita alterações inesperadas
e bugs no programa.



Mensagem-chave: Objetos imutáveis criam novas referências; objetos mutáveis podem ser alterados no mesmo objeto.





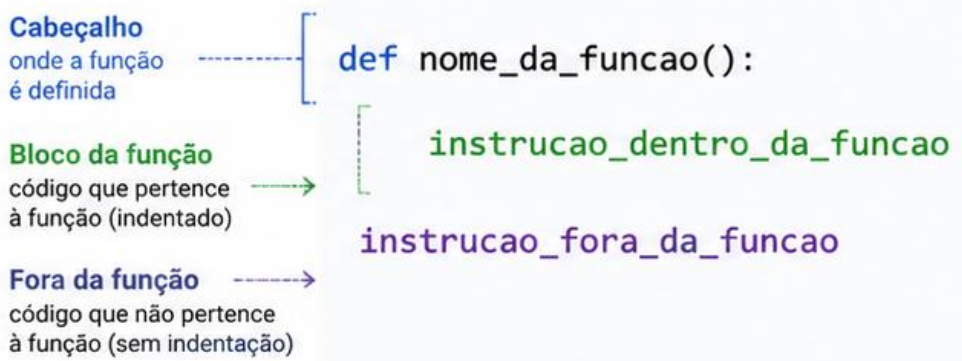
3. Criar funções com def



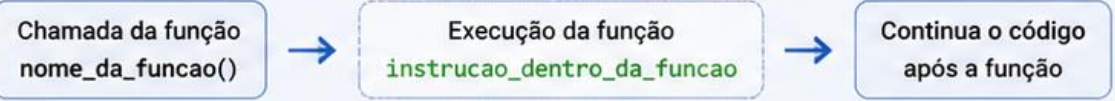
Ideia principal

Em Python, uma função é criada com def e o código que pertence à função é identificado pela indentação.

Estrutura geral



Exemplo com chamada



Elemento		Significado	
def	def	indica que vamos criar uma função	
()	()	local onde podem existir parâmetros	
:	:	marca o início do bloco	
	Indentação	indica o código que pertence à função	
	Fim da indentação	indica código fora da função	



Por que usar funções?

Reutilizar código, organizar o programa e evitar repetição.



Dica

A indentação define o que faz parte da função.



Lembre-se

Código bem indentado é mais legível e menos propenso a erros.



Boa prática

Use nomes descritivos para suas funções.



Mensagem-chave: O def define a função e a indentação determina o que pertence a ela.





4. Chamar uma função



Ideia principal

Criar uma função não significa executá-la. Para executar uma função, é necessário chamá-la pelo seu nome.

Definição e chamada

```
1 def mostrar_boas_vindas():
2     print("Esta linha está dentro da função.")
3
4 print("Esta linha está fora da função.")
5 mostrar_boas_vindas()
6
```

Saída esperada



Esta linha está fora da função.
Esta linha está dentro da função.



Executada fora da função



Executada dentro da função



Ordem importa!

Situação	Resultado
 Definir antes de chamar A função é definida primeiro e depois é chamada.	 correto O código funciona normalmente.
 Chamar antes de definir A função é chamada antes de ser definida.	 erro O Python não conhece a função ainda.



Dica

Sempre defina a função antes de chamá-la no código.



1. Definir a função



2. Chamar a função



3. Executar



Mensagem-chave: Para executar uma função, é necessário chamá-la pelo seu nome e na ordem correta.





5. Procedimentos vs funções



Ideia principal

Em Python, as funções são sempre criadas com `def`, mas podem ser usadas para executar uma ação ou devolver um valor.

 Utilização	 O que faz	 Resultado
 Procedimento	executa uma ação	normalmente usa <code>print()</code>
 Função	calcula ou processa dados	devolve um valor com <code>return</code>

Exemplo conceitual

1 Procedimento (executa uma ação)

```
# Procedimento
def mostrar_factorial(n):
    # calcular o factorial
    print(resultado)
```



Saída
(exemplo)
120

 Executa a ação dentro da função e exibe o resultado.

2 Função (devolve um valor)

```
# Função
def calcular_factorial(n):
    # calcular o factorial
    return resultado
```



Valor
(devolvido)
120

 Processa os dados e devolve o valor para ser usado onde quiser.



Como usar na prática



Procedimento

Chame apenas para gerar um efeito (ex.: exibir).



Função

Chame para obter um valor e usar em outras partes.



Benefício

Funções tornam o código mais reutilizável e testável.



Dica

Use `return` quando o resultado precisa ser aproveitado.



Mensagem-chave: Procedimentos fazem; funções calculam e devolvem.





6. print() vs return



Ideia principal

`print()` mostra informação no ecrã. `return` devolve um valor ao programa, permitindo guardar e reutilizar esse valor.

 Comando	 O que faz	 Utilização
<code>print()</code>	mostra informação no ecrã	apresentar resultados
<code>return</code>	devolve informação ao programa	guardar/reutilizar resultados

Consequência prática

Sem return (apenas print)

```
1 # sem return
2 valor = mostrar_factorial(5)
3 print(valor) # None
4
```

 Saída:
None



A função mostra o resultado, mas não o devolve.

Com return (devolve valor)

```
1 # com return
2 valor = calcular_factorial(5)
3 print(valor) # 120
4
```

 Saída:
120



A função devolve o resultado, que pode ser guardado e reutilizado.

Ideia essencial



Use `print()` para mostrar informação no ecrã.



Use `return` para devolver valores e reutilizá-los no código.



Dica rápida

Se o valor precisa ser usado depois, use `return`.

`print()`

→ mostra no ecrã



`return`

→ devolve para o programa



→ pode ser guardado e usado novamente



Mensagem-chave: `print()` exhibe; `return` devolve. Use a ferramenta certa para o objetivo certo.





7. Exemplo utilizado: factorial



Ideia principal

O factorial será usado para demonstrar a diferença entre procedimento com `print()` e função com `return`.

Fatorial



$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$



1. Procedimento (usa print())

```
1 def mostrar_factorial(n):
2     if n < 0:
3         print("Erro: n deve ser >= 0")
4         return
5     resultado = 1
6     for i in range(1, n + 1):
7         resultado *= i
8     print(resultado)
9
10 mostrar_factorial(5)      # saída: 120
11 mostrar_factorial(-3)    # saída: Erro
```



Saída no console:

```
120
Erro: n deve ser >= 0
```



2. Função (usa return)

```
1 def calcular_factorial(n):
2     if n < 0:
3         raise ValueError("n deve ser >= 0")
4     resultado = 1
5     for i in range(1, n + 1):
6         resultado *= i
7     return resultado
8
9 valor = calcular_factorial(5)    # valor recebe 120
10 print(valor)                   # saída: 120
11 # valor = calcular_factorial(-3) # gera erro
```



Saída no console:

```
120
```

Este exemplo permite mostrar:



Parâmetros

A função recebe entradas (n).



Return

A função devolve o valor calculado.



Validação

Verifica se n é um valor válido (≥ 0).



Tratamento de erros

Mostra como lidar com entradas inválidas.



O foco da aula está no **uso da função**, não no detalhe matemático.



Mensagem-chave: Usamos `print()` para exibir informações e `return` para devolver valores e reutilizá-los no programa.





8. Reutilizar o valor devolvido



Ideia principal

Quando uma função usa **return**, o valor devolvido pode ser guardado numa variável e usado noutros cálculos.

Exemplo de código

```
1 valor = calcular_factorial(5)
2
3 dobro = valor * 2
4 soma = valor + 10
5
6 print("Factorial:", valor)
7 print("Dobro:", dobro)
8 print("Somado a 10:", soma)
9
10 # Saída esperada:
    # Factorial: 120
    # Dobro: 240
    # Somado a 10: 130
```

Como funciona



Benefício

Reutilizar valores torna o código mais eficiente, organizado e fácil de manter.



Dica rápida

Use **return** quando precisar guardar o resultado para usar mais tarde no seu programa.

Exemplos de reutilização



Operações matemáticas



Cálculos estatísticos



Filtragem e processamento



Geração de relatórios



9. Parâmetros e argumentos



Ideia principal

As funções podem receber valores, permitindo executar a **mesma tarefa** com **dados diferentes**.

 Conceito	 Explicação	 Exemplo
 Parâmetro	nome usado na definição da função	<code>n</code> em <code>def calcular_factorial(n):</code>
 Argumento	valor passado quando a função é chamada	<code>5</code> em <code>calcular_factorial(5)</code>

Mesmo código, valores diferentes

```
1 print("Fatorial de 3:", calcular_factorial(3))
2 print("Fatorial de 5:", calcular_factorial(5))
3 print("Fatorial de 7:", calcular_factorial(7))
```



Saída esperada

```
Fatorial de 3: 6
Fatorial de 5: 120
Fatorial de 7: 5040
```



Por que usar?

Permite **reutilizar** a **mesma** função para diferentes entradas, tornando o código mais flexível e **reutilizável**.



Dica rápida

Os parâmetros ficam na definição da função; os argumentos são os valores passados na chamada.



Definição da função
(parâmetros)



Chamada da função
(argumentos)



Execução



Resultado

Exemplo:

```
def calcular_factorial(n):
    ...
    calcular_factorial(5)
```



10. Validação e tratamento de erros



Ideia principal

Uma função deve **verificar** se recebeu valores válidos **antes** de executar o cálculo.

 Momento	 O que acontece
 Validar	usar if para verificar os dados recebidos
 Devolver erro simples	usar return para devolver uma mensagem
 Lançar exceção	usar raise para sinalizar erro profissionalmente
 Tratar exceção	usar try/except onde a função é chamada

Exemplo prático: função com validação



```
def calcular_fatorial(n):  
    if not isinstance(n, int):  
        raise TypeError("n deve ser inteiro")  
    if n < 0:  
        raise ValueError("n deve ser >= 0")  
    resultado = 1  
    for i in range(2, n + 1):  
        resultado *= i  
    return resultado
```

Como tratar na chamada

```
try:  
    n = int(input("Digite n: "))  
    f = calcular_fatorial(n)  
    print(f"Fatorial de {n} = {f}")  
except (TypeError, ValueError) as e:  
    print(f"Erro: {e}")
```



Por que é importante?

- Evita que o programa quebre.
- Fornece mensagens claras ao usuário.
- Torna o código mais confiável e profissional.
- Facilita a manutenção e o debug.

Regras e exceções recomendadas

 Regra (validação)	 Exceção adequada	 Quando usar
1 n deve ser inteiro	TypeError	Quando o tipo de dado está incorreto.
2 n deve ser >= 0	ValueError	Quando o valor está fora do intervalo permitido.



Dica rápida

Valide primeiro, lance exceções quando necessário e trate-as na chamada. ✓

Validação simples vs exceções

✓ Validação simples

```
def calcular_fatorial(n):  
    if type(n) != int:  
        return "Erro: deve ser inteiro"  
    if n < 0:  
        return "Erro: deve ser >= 0"  
    return resultado
```

calculaFatorial

 Neste caso, a função devolve mensagens de erro como texto.

! Validação com exceções

```
def calcular_fatorial(n):  
    if type(n) != int:  
        raise TypeError("Deve ser inteiro")  
    if n < 0:  
        raise ValueError("Deve ser >= 0")  
    return resultado
```

calculaFatorial

 Neste caso, a função interrompe a execução e sinaliza o erro.

Abordagem		Vantagem	Limitação
	return "Erro" A função devolve uma mensagem de erro como texto.	 fácil de perceber Simples de implementar e entender para iniciantes.	 mistura números e mensagens O retorno pode ser um número (int) ou uma mensagem (str). Pode causar erros se não for verificado.
	raise A função lança uma exceção quando os dados são inválidos.	 mais profissional Separa erros do resultado esperado. Facilita a manutenção e o debug.	 exige try/except O código que chama a função precisa tratar as exceções.

Tratamento com try / except



Ideia principal

A função lança a exceção com **raise**; o código que chama a função pode tratar essa exceção com **try/except**.

Exemplo

```
1  valores_teste = [5, -3, 2.5]
2
3  for valor in valores_teste:
4      try:
5          resultado = calcular_fatorial(valor)
6          print("Fatorial de", valor, "::", resultado)
7      except ValueError as erro:
8          print("Erro de valor:", erro)
9      except TypeError as erro:
10         print("Erro de tipo:", erro)
11
12
```

Saída possível (exemplo)

>_ Saída

```
Fatorial de 5 : 120
Erro de valor: Deve ser >= 0
Erro de tipo: Deve ser inteiro
```



- O bloco **try** tenta executar o código que pode falhar.
- Se ocorrer um erro, o bloco **except** apropriado é executado.
- O programa continua com os próximos valores.



Como funciona



Chama a função
(com possíveis erros)



Se der certo:
executa e imprime
o resultado



Se der erro:
lança exceção
(raise)



try/except
trata o erro e
mostra a mensagem



O programa
continua

11. Mutabilidade dentro das funções

 **Ideia principal**
Quando um objeto é passado para uma função, o comportamento depende de o objeto ser **mutável** ou **imutável**.

 **Objeto imutável**
números, strings, tuplas, booleanos...

```
def alterar_numero(x):  
    x = x + 1  
    print("Dentro:", x)
```

```
numero = 10  
alterar_numero(numero)  
print("Fora:", numero) # 10
```

 **O que acontece?**
O valor é copiado para a função. Alterações não afetam a variável original fora da função.

 **Objeto mutável**
listas, dicionários, conjuntos...

```
def adicionar_nome(lista):  
    lista.append("Maria")  
    print("Dentro:", lista)
```

```
nomes = ["Ana", "João"]  
adicionar_nome(nomes)  
print("Fora:", nomes) # ['Ana', 'João', 'Maria']
```

 **O que acontece?**
A função modifica o próprio objeto (referenciado).
A mudança é visível fora da função.

 **Resumo rápido**
Imutável = não muda fora da função | **Mutável** = pode mudar fora da função

Exemplos de objetos

Imutáveis

123

"abc"

(1, 2)

Mutáveis

[]

{ }

{ }



12. Boas práticas



Ideia principal

Uma boa função deve ter um objetivo claro, um nome compreensível e uma única responsabilidade principal.



Boa prática



Nomes claros



Responsabilidade única



Simplicidade



Validar parâmetros



Evitar efeitos inesperados



Usar return



Ideia essencial

o nome deve indicar a tarefa

uma função deve fazer uma tarefa principal

deve ser fácil de ler, testar e corrigir

verificar dados quando houver risco

cuidado com objetos mutáveis

quando o resultado deve ser reutilizado



Resumo rápido



Nome bom =
código fácil de entender



Uma função,
um objetivo



Simples hoje,
fácil de manter amanhã



Valide antes,
evite erros depois



Return quando
o resultado importa



funções recursivas



Ideia principal

Uma função recursiva é uma função que se chama a si própria.
Nesta aula, a recursividade é apenas referida como **possibilidade**.

Exemplo simples

```
def factorial_recursivo(n):  
    if n == 0:  
        return 1  
    return n * factorial_recursivo(n - 1)
```

Como funciona (n = 4)

factorial_recursivo(4) = 4 * factorial_recursivo(3) → factorial_recursivo(3) = 3 * factorial_recursivo(2) → factorial_recursivo(2) = 2 * factorial_recursivo(1) → factorial_recursivo(1) = 1 * factorial_recursivo(0) → factorial_recursivo(0) = 1 (caso base)

Exemplo de uso

```
>>> factorial_recursivo(5)  
120  
  
>>> factorial_recursivo(0)  
1
```



Pontos importantes

- É necessário ter um **caso base** (aqui: `n == 0`) para a recursão parar.
- Cada chamada reduz o problema até chegar ao caso base.
- Recursão é elegante, mas pode ser menos eficiente que a iteração.
- Para valores muito grandes, pode causar estouro de pilha (`RecursionError`).





13. Funções lambda



Ideia principal

Uma função lambda é uma função curta, anônima e escrita normalmente numa única linha.

Estrutura e exemplo

```
# lambda argumentos: expressão  
  
lambda x: x * 2  
  
dobro = lambda x: x * 2  
  
print(dobro(5)) # 10
```



- **lambda** cria uma função anônima.
- Não precisa de **def** nem de **return**.
- Serve para expressões simples em uma única linha.

	Quando usar	Exemplo
	operações simples	multiplicar por 2 <code>lambda x: x * 2</code>
	funções curtas	expressões numa linha <code>lambda a, b: a + b</code>
	uso temporário	map() ou filter() <code>list(map(lambda x: x * 2, [1, 2, 3]))</code>



Dicas rápidas



Use **lambda** para coisas simples.



Prefira **def** para funções maiores e mais claras.



Leia sempre: **argumentos** antes dos dois-pontos, depois a **expressão**.



Fica muito útil com funções embutidas como **map()**, **filter()** e **sorted()**.

14. lambda com map() e filter()



Ideia principal

lambda define uma operação simples. **map()** transforma valores; **filter()** seleciona valores.

map(): transformar

```
numeros = [1, 2, 3, 4, 5]
dobros = list(map(lambda x: x * 2, numeros))
print(dobros) # [2, 4, 6, 8, 10]
```



map() aplica a função a todos os elementos e devolve um iterador. Por isso, usamos **list()**.

filter(): selecionar

```
numeros = [1, 2, 3, 4, 5, 6]
pares = list(filter(lambda x: x % 2 == 0, numeros))
print(pares) # [2, 4, 6]
```



filter() mantém apenas os elementos para os quais a função devolve **True**.

Conceito	Exemplo
 Função anónima	<code>lambda x: x * 2</code>
 Função de ordem superior	<code>map()</code> e <code>filter()</code>
 Programação funcional	transformar ou filtrar dados



Paradigmas de programação em Python



Python é uma **linguagem multiparadigma**: permite escrever código por passos, com funções, com objectos, com módulos e com execução assíncrona.

1

Procedimental



Definição: organiza o programa como uma sequência de instruções

Exemplo: for, if, variáveis

2

Funcional



Definição: aplica funções aos dados para transformar, seleccionar ou agregar

Exemplo: lambda, map(), filter(), reduce()

3

Orientado a objectos



Definição: organiza o programa em classes e objectos

Exemplo: class Animal:

4

Modular



Definição: divide o programa em módulos reutilizáveis

Exemplo: import math

5

Assíncrono



Definição: executa tarefas que esperam por eventos sem bloquear o programa

Exemplo: async, await

6

Orientado a eventos



Definição: reage a acontecimentos como cliques, mensagens ou pedidos

Exemplo: interfaces gráficas, servidores web



Problema



escolha do paradigma



código mais organizado



Mensagem-chave: Python permite escolher o paradigma mais adequado ao problema.



15. Programação funcional vs programação procedimental



Ideia principal

A programação procedimental organiza o programa como uma sequência de instruções.
A programação funcional organiza o processamento como a aplicação de funções aos dados.



Programação procedimental



Foco: passos e instruções



Usa frequentemente: for, if, variáveis auxiliares



Estilo: “fazer isto, depois aquilo”



Exemplo de código:

```
dobros = []  
for numero in numeros:  
    dobros.append(numero * 2)
```



Dados



Regra



Resultado

passos explícitos



Programação funcional



Foco: transformação de dados



Usa frequentemente: lambda, map(), filter(), reduce()



Estilo: “aplicar uma regra aos dados”



Exemplo de código:

```
dobros = list(map(lambda x: x * 2, numeros))
```



Dados



Regra



Resultado

funções aplicadas aos dados



1. Python introdutório

normalmente começa por programação procedimental.



2. Dados

programação funcional é útil para transformar e filtrar dados.



3. Pipelines

permite encadear operações.



4. Big Data

a mesma lógica aparece em Apache Spark.



Procedimental descreve passos; funcional descreve transformações sobre dados.

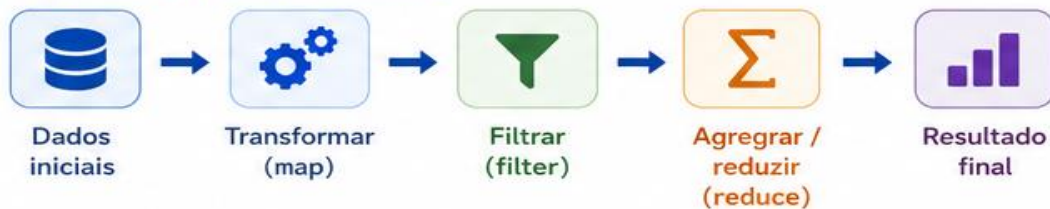
15. Programação funcional e funções lambda



Ideia principal

As funções lambda permitem aplicar regras simples aos dados. Em Python, são frequentemente usadas com funções como `map()`, `filter()` e `reduce()`, que representam operações típicas da programação funcional e são úteis para criar pipelines de processamento de dados.

Pipeline de processamento



`map()`: transformar

- Aplica uma função a todos os elementos.
- Devolve uma nova sequência transformada.

Exemplo: dobrar todos os números

[1, 2, 3, 4, 5] → [2, 4, 6, 8, 10]



`filter()`: selecionar

- Mantém apenas os elementos que cumprem uma condição.
- Devolve uma nova sequência filtrada.

Exemplo: manter apenas números pares

[1, 2, 3, 4, 5, 6] → [2, 4, 6]



`reduce()`: combinar

- Combina (agrega) os elementos de uma sequência usando uma função.
- Devolve um único valor final.

Exemplo: somar todos os números

[1, 2, 3, 4, 5] → 15



Exemplo em Python

```
from functools import reduce

numeros = [1, 2, 3, 4, 5]

dobros = list(map(lambda x: x * 2, numeros))
pares = list(filter(lambda x: x % 2 == 0, numeros))
soma = reduce(lambda x, y: x + y, numeros)

print(dobros)    # [2, 4, 6, 8, 10]
print(pares)     # [2, 4]
print(soma)      # 15
```

Linha	O que acontece
<code>map(lambda x: x * 2, numeros)</code>	Multiplica cada número por 2.
<code>filter(lambda x: x % 2 == 0, numeros)</code>	Mantém apenas os números pares.
<code>reduce(lambda x, y: x + y, numeros)</code>	Soma todos os números até obter um único valor.

Resultado esperado

[2, 4, 6, 8, 10]

[2, 4]

15



Uso em Big Data: Apache Spark

Em Spark, aplicamos a mesma lógica a grandes volumes de dados (distribuídos em vários computadores).

```
resultado = (
    rdd
    .map(lambda x: x * 2)
    .filter(lambda x: x % 2 == 0)
    .reduce(lambda x, y: x + y)
)
```

Exemplo de pipeline



Contexto	Utilização
Python básico	Aplicar regras a listas pequenas.
Análise de dados	Transformar, filtrar e agregar dados.
Big Data com Spark	Aplicar operações semelhantes a grandes volumes de dados distribuídos.



Mensagem-chave: `map()` transforma, `filter()` seleciona e `reduce()` combina vários valores num único resultado.

Estas operações permitem construir pipelines eficientes para processamento de dados, desde Python até Big Data com Spark.

15. Diferença entre def e lambda



Ideia principal

Tanto def como lambda permitem criar funções, mas são usados em contextos diferentes.

Aspeto	def	lambda
 Uso principal	funções normais e reutilizáveis	funções curtas e pontuais
 Tamanho	várias instruções	uma expressão
 Nome	normalmente tem nome definido	pode ser anónima
 Validação	adequado	não adequado
 Exemplo	calcular factorial	multiplicar por 2

Mesma operação simples

Com def

```
def dobrar(x):  
    return x * 2
```

Com lambda

```
lambda x: x * 2
```



O que é uma biblioteca em Python?



Uma biblioteca é um conjunto de código reutilizável. Em Python, podemos usar **bibliotecas já existentes** ou **criar os nossos próprios módulos** para organizar melhor o código e construir programas mais complexos.



Usar bibliotecas existentes: funções e ferramentas já criadas.



Criar módulos próprios: organizar o nosso código.

1 Código reutilizável



Funções, classes, módulos e ferramentas reutilizáveis.



2 Importação



Usamos `import` para aceder ao código.



3 Utilização



Usamos funções já existentes ou funções criadas por nós no nosso programa.



4 Organização



Usamos módulos próprios para organizar melhor o código em programas maiores.



Exemplos de bibliotecas e módulos

Exemplo	Utilização
 <code>math</code>	cálculos matemáticos
 <code>random</code>	valores aleatórios
 <code>datetime</code>	datas e horas

Exemplo	Utilização
 <code>modelo.py</code>	organizar dados e lógica do programa
 <code>vista.py</code>	organizar a apresentação dos resultados
 <code>controlador.py</code>	coordenar a execução do programa



Bibliotecas e módulos permitem reutilizar código e organizar melhor programas mais complexos.

Principais bibliotecas em Python para iniciantes

Bibliotecas úteis para começar a programar e explorar dados em Python



math

Permite realizar operações matemáticas.



Funções principais:

`sqrt()`, `pow()`, `ceil()`, `floor()`, `pi`



random

Permite gerar valores aleatórios.



Funções principais:

`randint()`, `choice()`, `shuffle()`, `random()`



datetime

Permite trabalhar com datas e horas.



Funções principais:

`datetime.now()`, `date.today()`, `strftime()`



statistics

Permite calcular medidas estatísticas simples.



Funções principais:

`mean()`, `median()`, `mode()`, `stdev()`



pandas

Permite trabalhar com dados em tabelas.



Funções principais:

`read_csv()`, `read_excel()`, `head()`, `describe()`



matplotlib.pyplot

Permite criar gráficos simples.



Funções principais:

`plot()`, `bar()`, `scatter()`, `show()`



As bibliotecas permitem reutilizar código já existente e alargar as capacidades dos programas em Python.



Formas de importar bibliotecas em Python



Ideia principal

Em todos os exemplos usamos a mesma função, `math.sqrt()`, para calcular a raiz quadrada de 25. O que muda é apenas a forma de importar.



1. Importar a biblioteca completa

```
import math  
  
print(math.sqrt(25))
```



Usamos o nome da biblioteca antes da função.



2. Importar apenas a função

```
from math import sqrt  
  
print(sqrt(25))
```



A função pode ser usada diretamente.



3. Importar com alias

```
import math as m  
  
print(m.sqrt(25))
```



Damos um nome mais curto à biblioteca.



4. Importar a função com alias

```
from math import sqrt as raiz  
  
print(raiz(25))
```



Damos um nome alternativo à função.

Forma		Quando é útil
	<code>import math</code>	quando queremos usar várias funções da biblioteca
	<code>from math import sqrt</code>	quando precisamos apenas de uma função
	<code>import math as m</code>	quando o nome da biblioteca é usado muitas vezes
	<code>from math import sqrt as raiz</code>	quando queremos um nome mais claro ou mais curto



Resultado pedagógico: o aluno percebe que a função é a mesma; muda apenas a sintaxe de importação.

De bibliotecas existentes a módulos próprios

Bibliotecas permitem reutilizar código e, em Python, também podemos criar os nossos próprios módulos para organizar programas mais complexos.



Ideia principal: Usar bibliotecas em Python está ligado à programação modular. Além de importar bibliotecas existentes, também podemos criar os nossos próprios módulos para organizar programas mais complexos.



Bibliotecas existentes
funções e ferramentas já criadas.



Módulos próprios
código criado por nós para organizar o programa.

1 O que é programação modular?

	Módulo	unidade de código reutilizável (criada por nós ou de terceiros)
	Biblioteca	conjunto de módulos com funcionalidades já desenvolvidas
	Importação	forma de utilizar módulos ou bibliotecas num programa
	Reutilização	uso de código existente sem reescrever



2 Bibliotecas existentes vs módulos próprios

Bibliotecas existentes (de terceiros)		Módulos próprios (criados por nós)	
<code>import math</code>	usa uma biblioteca já existente	<code>import modelo</code>	usa o módulo de dados e lógica
<code>import random</code>	reutiliza funcionalidades prontas	<code>import vista</code>	usa o módulo de apresentação
<code>from datetime import datetime</code>	importa apenas a parte necessária	<code>import controlador</code>	usa o módulo de coordenação
<code>import numpy as np</code>	usa um módulo com alias para simplificar o código	<code>import utils</code>	usa funções auxiliares criada por nós

3 Exemplo: organização em módulos (exemplo MVC)

projeto_notas/	
	<code>modelo.py</code>
	<code>vista.py</code>
	<code>controlador.py</code>
	<code>main.py</code>

Módulo	Responsabilidade
 <code>modelo.py</code>	dados, validações e cálculos
 <code>vista.py</code>	apresentação dos resultados (interfaces e saída)
 <code>controlador.py</code>	coordena a execução e liga o modelo à vista
 <code>main.py</code>	ponto de entrada do programa

4 Vantagens da programação modular com bibliotecas e módulos próprios



Organização
o código fica dividido em partes com responsabilidades claras.



Reutilização
podemos usar os mesmos módulos em vários programas.



Manutenção
é mais fácil corrigir ou melhorar apenas uma parte do programa.



Produtividade
desenvolvemos programas mais rapidamente e com menos erros.



A programação modular permite transformar um programa grande e misturado em módulos pequenos, reutilizáveis e fáceis de compreender.



Problema a resolver: gestão de notas



Queremos criar um programa que analisa as notas de vários alunos, valida os dados, calcula resultados e apresenta mensagens no ecrã.



Dados dos alunos



Validação das notas



Cálculos



Classificação



Apresentação



O programa deve

- guardar nomes e notas
- verificar notas inválidas
- calcular média, máxima e mínima
- atribuir classificação
- mostrar resultados ou erros



Porque é um bom exemplo?

- tem várias responsabilidades
- usa funções, listas e dicionários
- usa validação e exceções
- prepara a divisão em módulos



Quando um programa começa a ter várias responsabilidades, faz sentido organizá-lo em módulos.

Exemplo completo num único ficheiro



O programa funciona, mas junta dados, validações, cálculos, apresentação, controlo e síntese final no mesmo ficheiro.

Dados + Modelo

```
alunos = {"Ana": [14, 16, 15], "João": [9, 11, 10],
          "Maria": [18, 17, 19], "Pedro": [12, 25, 13]}

def validar_nota(nota):
    if type(nota) != int and type(nota) != float:
        raise TypeError("A nota deve ser um número.")
    if nota < 0 or nota > 20:
        raise ValueError("A nota deve estar entre 0 e 20.")

def calcular_media(notas):
    for nota in notas:
        validar_nota(nota)
    return sum(notas) / len(notas)

def classificar_media(media):
    if media >= 16:
        return "Muito bom"
    elif media >= 10:
        return "Aprovado"
    else:
        return "Reprovado"
```

Vista + Controlador

```
def mostrar_resultado(nome, media, classificacao):
    print("Aluno:", nome)
    print("Média:", media)
    print("Classificação:", classificacao)

def mostrar_sintese(incluidos, excluidos, media_final):
    print(" Incluídos:", incluidos)
    print(" Excluídos:", excluidos)
    print(" Média final:", media_final)

medias_validas = []
alunos_incluidos = []
alunos_excluidos = []

for nome, notas in alunos.items():
    try:
        media = calcular_media(notas)
        classificacao = classificar_media(media)
        medias_validas.append(media)
        alunos_incluidos.append(nome)
        mostrar_resultado(nome, media, classificacao)
    except ValueError as erro:
        alunos_excluidos.append(nome)

media_final = calcular_media(medias_validas)
mostrar_sintese(alunos_incluidos, alunos_excluidos, media_final)
```



Dados



Validação



Cálculo



Inclusão/Exclusão



Média final



Apresentação



O programa funciona, mas já mostra a necessidade de separar responsabilidades em módulos.

Exemplo inicial: programa num único ficheiro



Antes de dividir o programa em módulos, criamos a primeira versão com tudo no mesmo ficheiro.



O programa já faz várias tarefas

- guardar dados
- validar notas
- calcular resultados
- classificar alunos
- incluir/excluir alunos
- calcular média final
- mostrar mensagens



Dados de exemplo

```
alunos = {  
    "Ana": [14, 16, 15],  
    "João": [9, 11, 10],  
    "Maria": [18, 17, 19],  
    "Pedro": [12, 25, 13]  
}
```



Execução principal

```
medias_validas = []  
alunos_incluidos = []  
alunos_excluidos = []  
for nome, notas in alunos.items():  
    try:  
        media = calcular_media(notas)  
        medias_validas.append(media)  
        alunos_incluidos.append(nome)  
        mostrar_resultado(...)  
    except ValueError as erro:  
        alunos_excluidos.append(nome)  
        mostrar_erro(...)  
  
mostrar_sintese(alunos_incluidos, alunos_excluidos)  
  
media_final = calcular_media(medias_validas)
```



Problema

O código funciona, mas várias responsabilidades continuam misturadas no mesmo ficheiro.

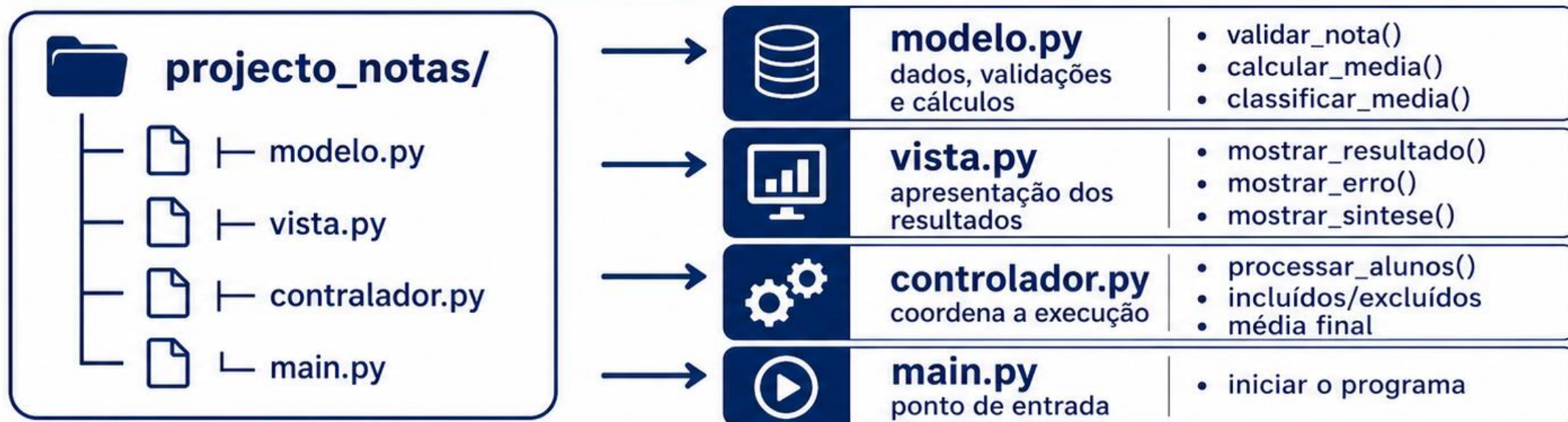


Mensagem-chave: a primeira versão funciona, mas mostra a necessidade de separar responsabilidades em módulos.

Estrutura modular do mesmo programa



Depois da primeira versão num único ficheiro, o programa pode ser dividido em módulos. Cada ficheiro fica responsável por uma parte específica do programa.



Antes		Depois	
	tudo no mesmo ficheiro		ficheiros separados
	responsabilidades misturadas		responsabilidades claras
	mais difícil de manter		mais fácil de reutilizar



A programação modular transforma um programa grande em módulos pequenos, reutilizáveis e fáceis de compreender.

Os três módulos do programa



Depois de separar o programa, cada módulo fica responsável por uma parte específica: calcular, apresentar ou coordenar.



modelo.py

dados, validações e cálculos

- validar_nota()
- calcular_media()
- obter_nota_maxima()
- obter_nota_minima()
- classificar_media()



vista.py

apresentação dos resultados

- mostrar_resultado()
- mostrar_erro()
- mostrar_alunos_incluidos_e_excluidos()
- mostrar_media_turma()



controlador.py

coordena a execução

- processar_alunos()
- chama o modelo
- chama a vista
- trata erros

modelo.py



controlador.py



vista.py

O controlador liga o modelo à vista



Cada módulo tem uma responsabilidade principal, tornando o código mais claro, reutilizável e fácil de manter.



main.py — programa principal



O ficheiro principal importa o controlador, define os dados iniciais e inicia a execução do programa.

main.py

```
1 from controlador import processar_alunos
2
3 alunos = {
4     "Ana": [14, 16, 15],
5     "João": [9, 11, 10],
6     "Maria": [18, 17, 19],
7     "Pedro": [12, 25, 13]
8 }
9 processar_alunos(alunos)
```

1



Importação

traz a função `processar_alunos()` do `controlador.py`

2



Dados iniciais

define nomes e listas de notas

3



Chamada principal

executa o processamento dos alunos



main.py



controlador.py



modelo.py



vista.py



O programa principal fica simples: prepara os dados e chama a função que coordena tudo.



modelo.py — dados, validações e cálculos



IDEIA PRINCIPAL

Este módulo contém a lógica principal do programa: valida notas, calcula resultados e classifica médias.

`</>` modelo.py

```
1  # modelo.py
2  # Responsável por validações, cálculos e classificação
3
4  def validar_nota(nota):
5      if type(nota) != int and type(nota) != float:
6          raise TypeError("A nota deve ser um número.")
7
8      if nota < 0 or nota > 20:
9          raise ValueError("A nota deve estar entre 0 e 20.")
10
11 def validar_lista_notas(notas):
12     if type(notas) != list:
13         raise TypeError("As notas devem estar numa lista.")
14
15     if len(notas) == 0:
16         raise ValueError("A lista de notas não pode estar vazia.")
17
18     for nota in notas:
19         validar_nota(nota)
20
21 def calcular_media(notas):
22     validar_lista_notas(notas)
23
24     soma = 0
25
26     for nota in notas:
27         soma = soma + nota
28
29     return soma / len(notas)
30
31 def obter_nota_maxima(notas):
32     validar_lista_notas(notas)
33
34     maior = notas[0]
35
36     for nota in notas:
37         if nota > maior:
38             maior = nota
39
40     return maior
41
42 def obter_nota_minima(notas):
43     validar_lista_notas(notas)
44
45     menor = notas[0]
46
47     for nota in notas:
48         if nota < menor:
49             menor = nota
50
51     return menor
52
53 def classificar_media(media):
54     if media >= 16:
55         return "Muito bom"
56     elif media >= 10:
57         return "Aprovado"
58     else:
59         return "Reprovado"
```



O modelo trata dos dados e da lógica de cálculo, sem apresentar resultados no ecrã.

vista.py – apresentação dos resultados



IDEIA PRINCIPAL

Este módulo mostra informação no ecrã: resultados, erros, alunos incluídos, alunos excluídos e média final.

```
1  # vista.py
2  # Responsável pela apresentação dos resultados
3
4  def mostrar_resultado(nome, media, maxima, minima, classificacao):
5      print("Aluno:", nome)
6      print("Média:", media)
7      print("Nota máxima:", maxima)
8      print("Nota mínima:", minima)
9      print("Classificação:", classificacao)
10     print()
11
12     def mostrar_erro(nome, erro):
13         print("Erro nos dados do aluno:", nome)
14         print("Detalhe:", erro)
15         print()
16
17     def mostrar_alunos_incluidos_e_excluidos(alunos_incluidos, alunos_excluidos):
18         print("Alunos incluídos no cálculo da média final:")
19
20         for aluno in alunos_incluidos:
21             print("-", aluno)
22
23         print()
24
25         print("Alunos excluídos do cálculo da média final:")
26
27         if len(alunos_excluidos) == 0:
28             print("- Nenhum aluno excluído")
29         else:
30             for aluno in alunos_excluidos:
31                 print("-", aluno)
32
33         print()
34
35     def mostrar_media_turma(media_turma):
36         print("Média final da turma com alunos válidos:", media_turma)
37
38     def mostrar_sem_alunos_validos():
39         print("Não existem alunos válidos para calcular a média final da turma.")
```



A vista apresenta informação, mas não valida dados nem calcula resultados.

controlador.py — coordenação da execução



IDEIA PRINCIPAL

Este módulo liga o modelo à vista: processa os alunos, trata erros, guarda incluídos e excluídos e calcula a média final.



controlador.py

```
1  # controlador.py
2  # Responsável por coordenar a execução do programa
3
4  from modelo import calcular_media
5  from modelo import obter_notas_maxima
6  from modelo import obter_notas_minima
7  from modelo import classificar_media
8
9  from vista import mostrar_resultado
10 from vista import mostrar_erro
11 from vista import mostrar_alunos_incluidos_e_excluidos
12 from vista import mostrar_media_turma
13 from vista import mostrar_sem_alunos_validos
14
15 def processar_alunos(alunos):
16     medias_validas = []
17     alunos_incluidos = []
18     alunos_excluidos = []
19
20     for nome, notas in alunos.items():
21         try:
22             media = calcular_media(notas)
23             maxima = obter_notas_maxima(notas)
24             minima = obter_notas_minima(notas)
25             classificacao = classificar_media(media)
26
27             medias_validas.append(media)
28             alunos_incluidos.append(nome)
29
30             mostrar_resultado(nome, media, maxima, minima, classificacao)
31         except TypeError as erro:
32             alunos_excluidos.append(nome)
33             mostrar_erro(nome, erro)
34
35     except ValueError as erro:
36         alunos_excluidos.append(nome)
37         mostrar_erro(nome, erro)
38
39     mostrar_alunos_incluidos_e_excluidos(alunos_incluidos, alunos_excluidos)
40
41     if len(medias_validas) > 0:
42         media_turma = calcular_media(medias_validas)
43         mostrar_media_turma(media_turma)
44     else:
45         mostrar_sem_alunos_validos()
```



O controlador decide o fluxo do programa: chama funções, trata exceções e coordena a execução.



main.py — programa principal



Ideia principal

Este ficheiro é o ponto de entrada:
importa o controlador, define os dados iniciais e inicia o programa.

```
</> main.py
1  # main.py
2  # Programa principal
3
4  from controlador import processar_alunos
5
6  alunos = {
7      "Ana": [14, 16, 15],
8      "João": [9, 11, 10],
9      "Maria": [18, 17, 19],
10     "Pedro": [12, 25, 13]
11 }
    processar_alunos(alunos)
```



O main.py fica simples: prepara os dados e chama a função que coordena tudo.